```
            thread[i].join();
    }
    throw;
}
```

Now this works. All the threads will be joined, no matter how the code leaves the block. However, `try-catch` blocks are ugly, and you have duplicate code. You're joining the threads both in the "normal" control flow *and* in the `catch` block. Duplicate code is rarely a good thing, because it means more places to change. Instead, let's extract this out into the destructor of an object; it is, after all, the idiomatic way of cleaning up resources in C++. Here's your class:

```
class join_threads
{
    std::vector<std::thread>& threads;
public:
    explicit join_threads(std::vector<std::thread>& threads_):
        threads(threads_)
    {}
    ~join_threads()
    {
        for(unsigned long i=0;i<threads.size();++i)
         {
            if(threads[i].joinable())
                threads[i].join();
         }
    }
};
```

This is similar to your `thread_guard` class from listing 2.3, except it's extended for the whole vector of threads. You can then simplify your code as follows.

**Listing 8.4. An exception-safe parallel version of `std::accumulate`**